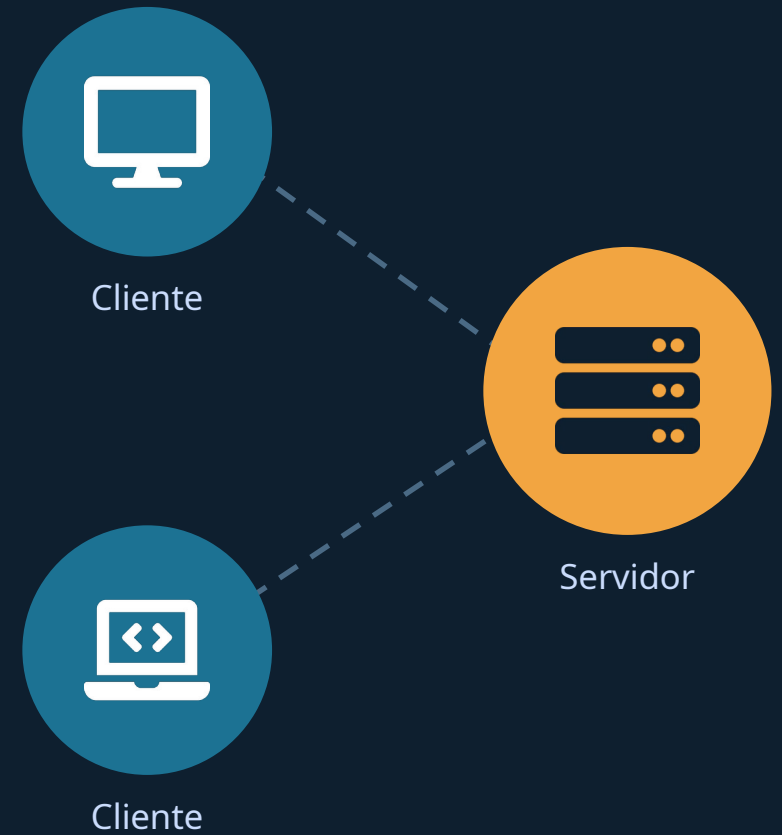


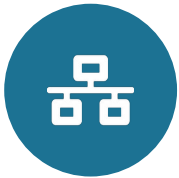
AULA 3 · DESENVOLVIMENTO DE SOFTWARE CLIENTE-SERVIDOR

Aplicações Cliente-Servidor

Comunicação em rede pela ótica da aplicação:
modelo cliente-servidor, sockets e uma POC em
Java



Onde estamos na disciplina



Aulas 1 e 2

Redes de computadores

Intercomunicação: como os dados viajam entre máquinas.

Concluído



Aula 3 · hoje

Aplicações em rede

Modelo cliente-servidor, sockets e uma POC em Java.

Hoje



Aula 4

Evolução da aplicação

Concorrência, protocolos mais ricos e interface gráfica.

Próxima

O que você vai aprender hoje

- 1 Explicar o modelo cliente-servidor e o papel de cada lado
- 2 Descrever o que é um socket e como ele identifica uma comunicação
- 3 Distinguir TCP e UDP do ponto de vista do desenvolvedor
- 4 Ler e executar uma aplicação Java simples com Socket e ServerSocket



AO FINAL DA AULA

**Um cliente e um
servidor Java
conversando por
socket**

Agenda da aula

01

Modelo cliente-servidor

Papéis, ciclo requisição-resposta, exemplos e arquiteturas.

Slides 5 a 11

02

Sockets e comunicação

Portas, TCP e UDP, classes Java e protocolos de aplicação.

Slides 12 a 20

03

Prática: POC em Java

Um servidor e um cliente de console, do código à execução.

Slides 21 a 28

Fechamento: resumo, limitações da POC e atividade proposta (slides 29 e 30)

Ponto de partida: o que a rede já nos dá

Aplicação

HTTP, SMTP, o seu protocolo

Transporte

TCP, UDP, portas

Rede

IP, roteamento

Enlace e física

quadros, sinais, meio

A rede já resolve

- Endereçar máquinas (IP)
- Entregar dados a um processo (portas, TCP e UDP)
- Transportar bits pelo meio físico

Falta à aplicação

- Definir o que as mensagens significam
- Decidir quem inicia e quem responde
- Escrever o código que usa a rede (sockets)

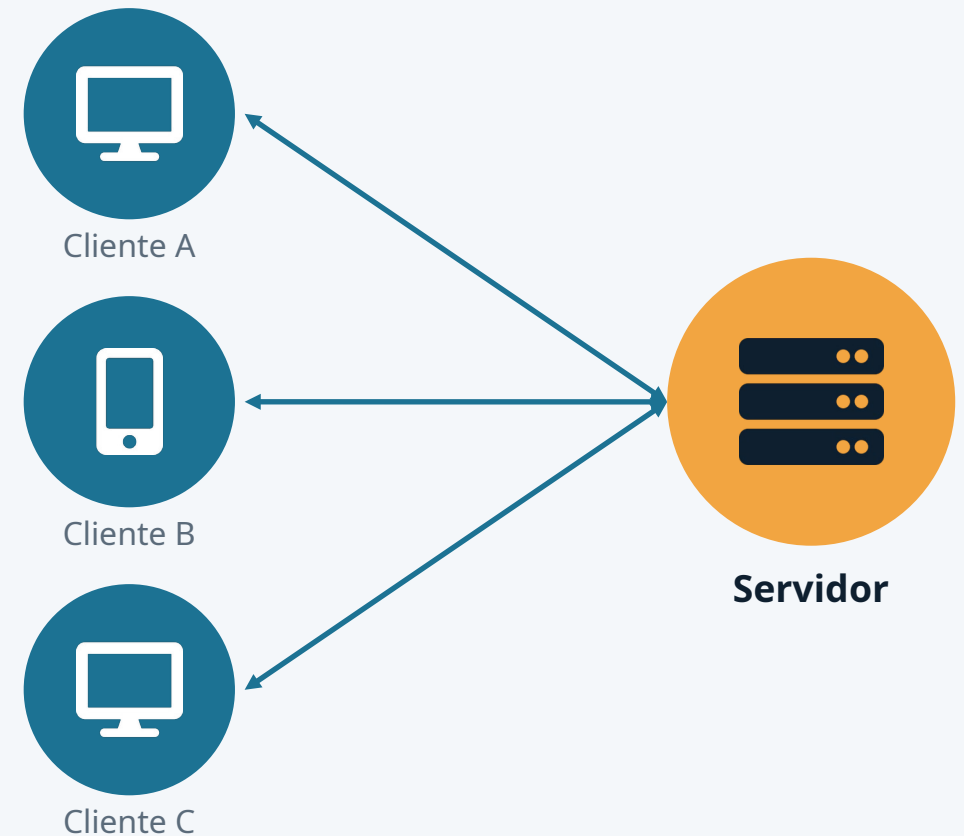
O que é o modelo cliente-servidor

Um modelo de distribuição de tarefas entre processos que se comunicam pela rede.

Servidor: oferece um serviço e aguarda pedidos

Cliente: solicita o serviço e usa a resposta

Papéis de processos, não de máquinas: o mesmo computador pode ter os dois



Papéis: cliente e servidor



Cliente

- Inicia a comunicação (papel ativo)
- Conhece o endereço e a porta do servidor
- Vida curta: conecta, pede, recebe e encerra
- Costuma interagir com o usuário

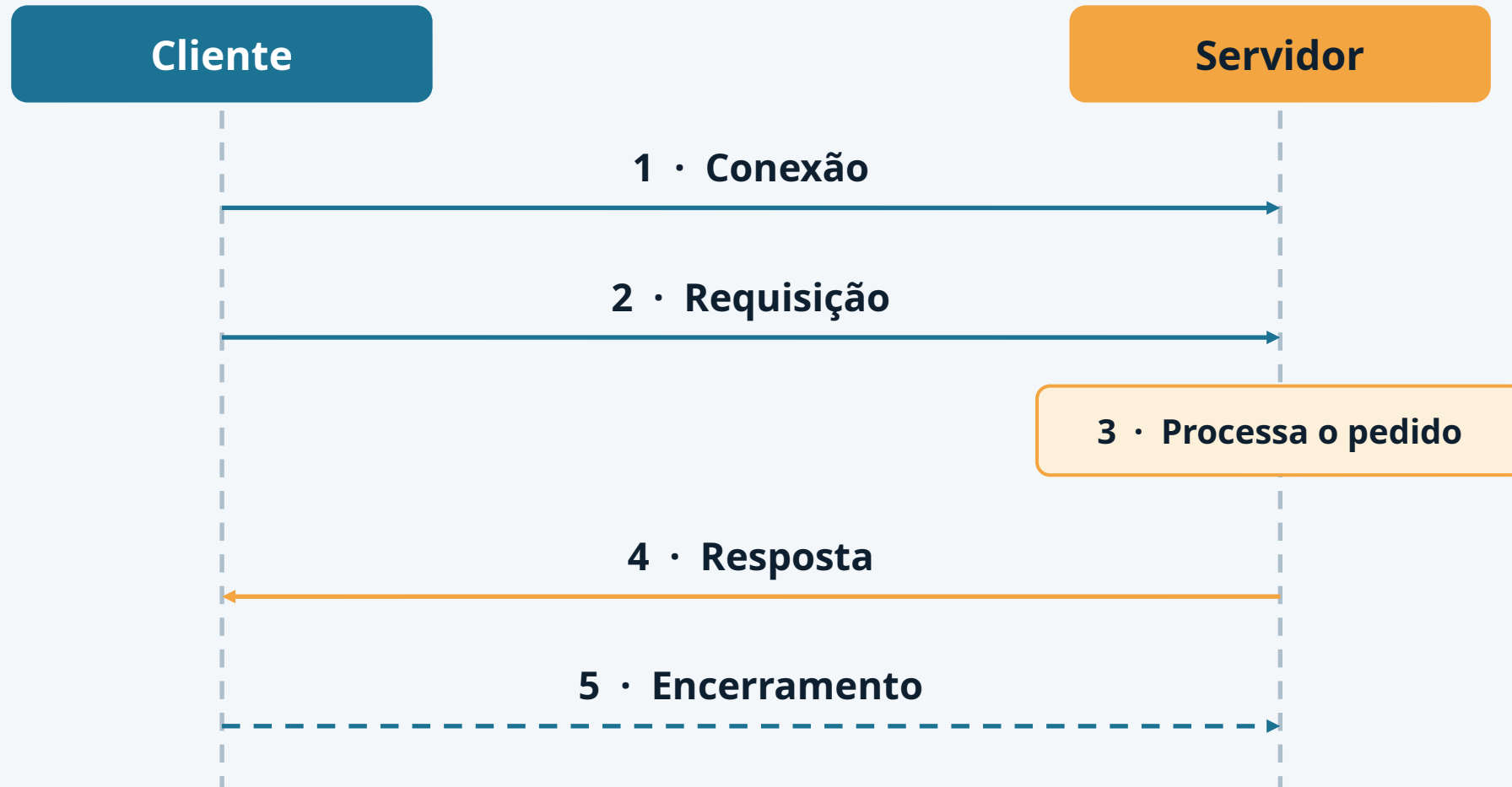


Servidor

- Aguarda conexões (papel passivo)
- Tem endereço e porta conhecidos
- Executa continuamente, à espera de pedidos
- Atende um ou vários clientes

Um mesmo programa pode ser cliente de um serviço e servidor de outro.

O ciclo requisição-resposta



Exemplos do dia a dia



Web

Navegador ↔ servidor web
(HTTP e HTTPS)



E-mail

Cliente de e-mail ↔ servidor
(SMTP, IMAP)



Banco de dados

Aplicação ↔ SGBD (por exemplo, via JDBC)



Jogos online

Jogo no dispositivo ↔ servidor da partida



Apps móveis

Aplicativo ↔ API hospedada no servidor

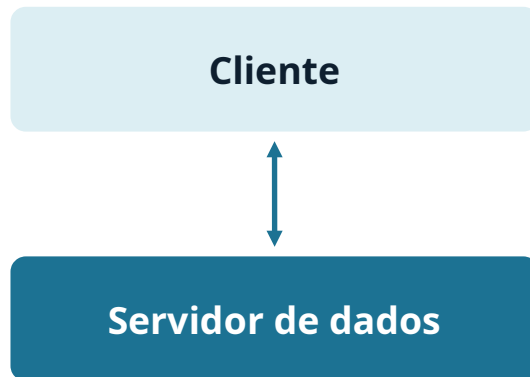


Acesso remoto

Terminal ↔ servidor SSH

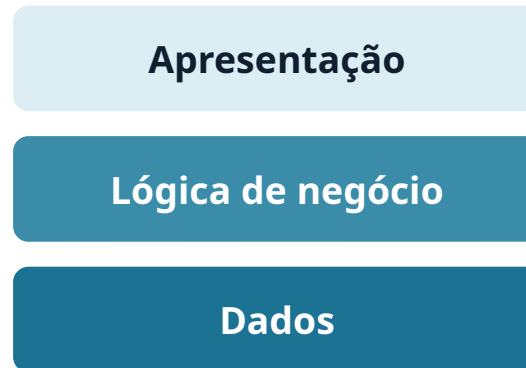
Arquiteturas: duas camadas, três camadas e P2P

Duas camadas



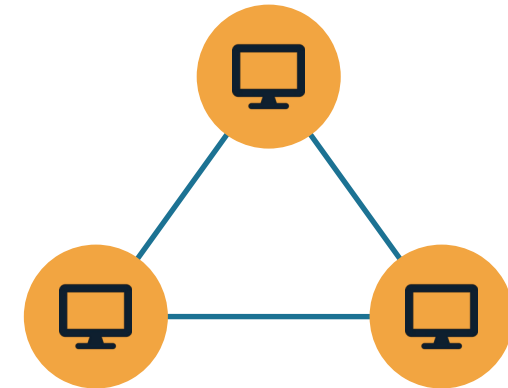
Ex.: aplicação desktop acessando o banco de dados diretamente.

Três camadas



Ex.: app web ou móvel que chama uma API, que acessa o banco de dados.

Ponto a ponto (P2P)



Cada nó é cliente e servidor ao mesmo tempo. Ex.: compartilhamento de arquivos.

Desafios de quem constrói aplicações em rede



Concorrência

Vários clientes ao mesmo tempo disputando o servidor.



Falhas de rede

Conexões caem e mensagens atrasam ou se perdem.



Estado da sessão

Lembrar (ou não) quem é o cliente entre uma requisição e outra.

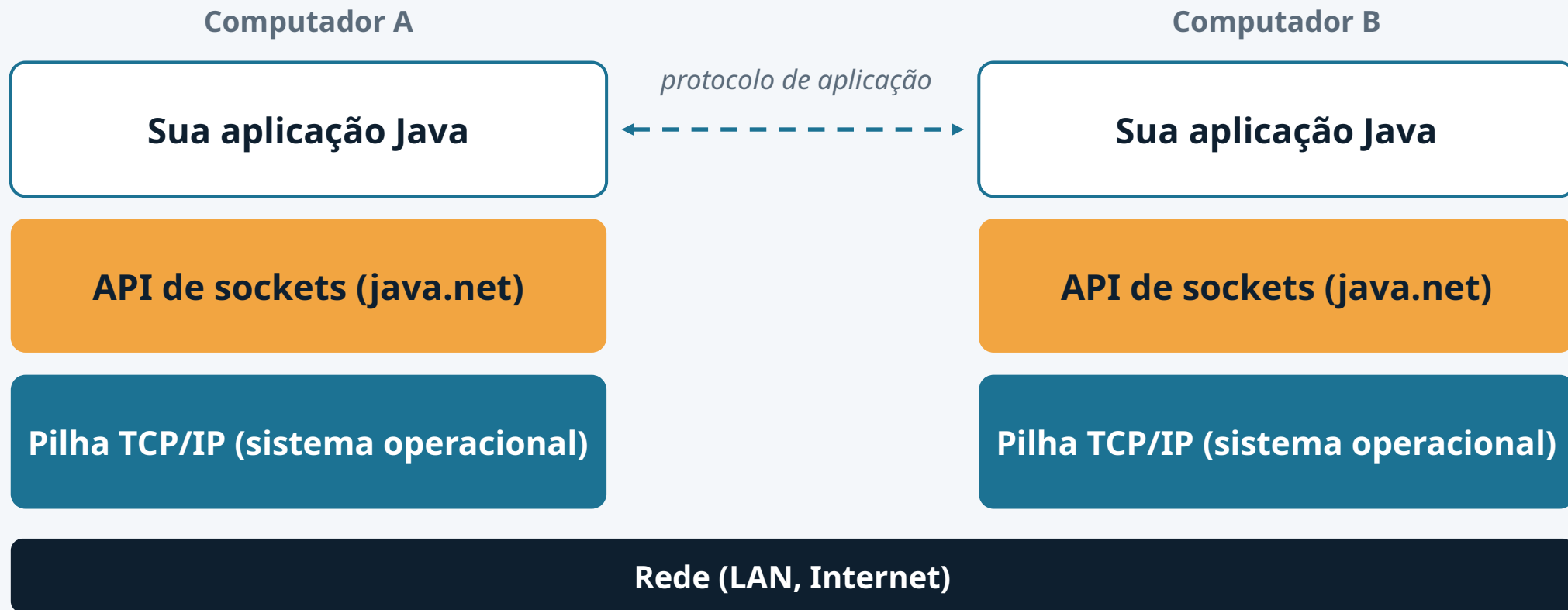


Segurança

Autenticação, criptografia e validação do que chega.

Hoje: uma conversa simples entre dois programas. Na aula 4: enfrentar esses desafios.

Da rede à aplicação: onde os sockets entram



O socket é a porta de entrada da aplicação para os serviços de rede do sistema operacional.

O que é um socket

DEFINIÇÃO

Socket é um ponto de extremidade de uma comunicação em rede.

endereço IP

+

porta

+

protocolo

Analogia

IP: o endereço do prédio

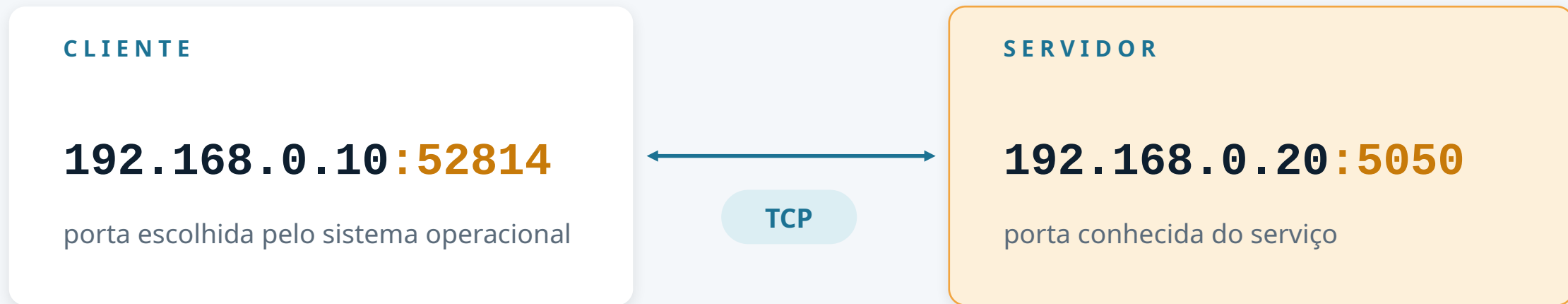
Porta: o número do apartamento

Socket: o ponto onde a conversa acontece

Para o programador

Um socket se comporta como um arquivo: abrir, ler, escrever e fechar.

Como uma conexão é identificada



Uma conexão TCP é identificada por cinco valores:

Protocolo

IP de origem

Porta de origem

IP de destino

Porta de destino

Dois clientes falando com o mesmo servidor geram conexões distintas: pelo menos um dos valores de origem muda.

Portas: o número do apartamento

0 – 1023

Bem conhecidas

80 HTTP
443 HTTPS
22 SSH
25 SMTP

Em muitos sistemas exigem privilégios de administrador.

1024 – 49151

Registradas

3306 MySQL
5432 PostgreSQL
8080 HTTP alternativa

Livres para aplicações comuns.

49152 – 65535

Dinâmicas (efêmeras)

Porta de origem temporária do cliente

Atribuídas pelo sistema operacional; a faixa exata varia.

Na POC: porta 5050, na faixa registrada e sem necessidade de privilégios.

TCP × UDP: duas formas de conversar

TCP

como uma ligação telefônica

- Orientado a conexão
- Entrega confiável e em ordem
- Controle de fluxo e de congestionamento
- Java: Socket e ServerSocket

UDP

como enviar cartões-postais

- Sem conexão
- Sem garantia de entrega ou de ordem
- Menos sobrecarga e menor latência
- Java: DatagramSocket e DatagramPacket

Nesta aula: TCP, pela confiabilidade e pela simplicidade de uso.

Ciclo de vida de uma conexão TCP

SERVIDOR (passivo)

CLIENTE (ativo)



Sockets em Java: as classes principais

ServerSocket

TCP · servidor

Escuta em uma porta. O método `accept()` bloqueia até um cliente chegar e devolve um `Socket`.

Socket

TCP · cliente e servidor

Extremidade de uma conexão. Oferece `getInputStream()` e `getOutputStream()`.

InetAddress

endereços

Representa um endereço IP ou nome de host. Ex.: `InetAddress.getByName("localhost")`.

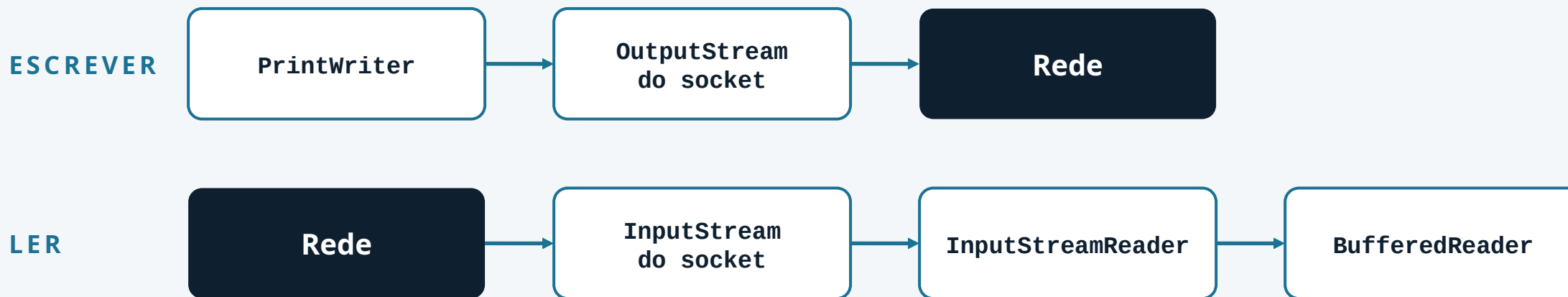
DatagramSocket

UDP · sem conexão

Envia e recebe datagramas (`DatagramPacket`) sem estabelecer conexão.

Pacotes: `java.net` (rede) e `java.io` (fluxos de entrada e saída)

Lendo e escrevendo: streams sobre o socket



println(msg) envia uma linha; com autoflush, sai na hora

readLine() bloqueia até chegar uma linha; retorna null se o outro lado fechou

try-with-resources fecha streams e socket automaticamente

Protocolo de aplicação: dando sentido aos bytes

O socket entrega bytes, não significado. O protocolo precisa responder:

- 1 Quem fala primeiro?
- 2 Qual é o formato das mensagens?
- 3 Como saber onde uma mensagem termina?
- 4 Como e quando encerrar a conversa?

Protocolo de aplicação: conjunto de regras que responde a essas perguntas.

EXEMPLO: HTTP

Cliente:

```
GET /index.html HTTP/1.1  
Host: www.exemplo.com.br
```

Servidor:

```
HTTP/1.1 200 OK  
Content-Type: text/html  
Content-Length: 1256
```

Regras claras permitem que programas escritos por pessoas diferentes se entendam.

A POC: escopo e funcionalidade

Funcionalidade

Cliente envia

ola mundo



Servidor devolve

OLA MUNDO

Eco em maiúsculas: uma única funcionalidade.

Dentro do escopo

- ✓ Um cliente por vez
- ✓ Aplicação de console
- ✓ TCP em localhost
- ✓ Protocolo de texto, uma mensagem por linha

Fora do escopo

- ✗ Vários clientes simultâneos
- ✗ Interface gráfica (JavaFX)
- ✗ Persistência e banco de dados
- ✗ Autenticação e criptografia

POC: prova de conceito. Valida que a ideia funciona, sem a preocupação de ser completa.

O protocolo da POC

Cliente envia	Servidor responde
Qualquer texto (uma linha)	O mesmo texto em maiúsculas
SAIR	TCHAU, e a conexão é encerrada

Regras

- Mensagens em texto, uma por linha (terminadas em quebra de linha)
- O cliente sempre fala primeiro
- Uma resposta para cada mensagem recebida

EXEMPLO DE SESSÃO

> **ola mundo**

Servidor: OLA MUNDO

> **sockets**

Servidor: SOCKETS

> **SAIR**

Servidor: TCHAU

Código: o servidor

```
import java.io.*;
import java.net.*;

public class Servidor {
    public static void main(String[] args) throws IOException {
        System.out.println("Aguardando cliente na porta 5050...");
        try (ServerSocket servidor = new ServerSocket(5050);
             Socket cliente = servidor.accept();
             BufferedReader entrada = new BufferedReader(
                 new InputStreamReader(cliente.getInputStream()));
             PrintWriter saida = new PrintWriter(cliente.getOutputStream(), true)) {
            System.out.println("Cliente: " + cliente.getInetAddress());
            String linha;
            while ((linha = entrada.readLine()) != null) {
                if (linha.equalsIgnoreCase("SAIR")) {
                    saida.println("TCHAU");
                    break;
                }
                saida.println(linha.toUpperCase());
            }
        }
    }
}
```

ARQUIVO

Servidor.java

COMPILAR

javac Servidor.java

EXECUTAR

java Servidor

O servidor, passo a passo

1

```
new ServerSocket(5050)
```

Reserva a porta 5050 e passa a escutar conexões.

2

```
servidor.accept()
```

Bloqueia até um cliente conectar e devolve um Socket exclusivo dessa conversa.

3

```
getInputStream() getOutputStream()
```

Os fluxos do socket são envolvidos por `BufferedReader` e `PrintWriter`.

4

```
entrada.readLine()
```

Espera uma linha completa; retorna null quando o cliente fecha a conexão.

5

```
saida.println(...)
```

Envia a resposta ao cliente; com `autoflush`, ela sai imediatamente.

Código: o cliente

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Cliente {
    public static void main(String[] args) throws IOException {
        try (Socket socket = new Socket("localhost", 5050);
            BufferedReader entrada = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter saida = new PrintWriter(socket.getOutputStream(), true);
            Scanner teclado = new Scanner(System.in)) {
            String msg;
            do {
                System.out.print("> ");
                msg = teclado.nextLine();
                saida.println(msg);
                System.out.println("Servidor: " + entrada.readLine());
            } while (!msg.equalsIgnoreCase("SAIR"));
        }
    }
}
```

ARQUIVO

Cliente.java

EXECUTAR

java Cliente
(em outro terminal)

Para testar entre máquinas, troque "localhost" pelo IP do servidor.

Executando a POC

TERMINAL 1 · SERVIDOR

```
$ javac Servidor.java Cliente.java
$ java Servidor
Aguardando cliente na porta 5050...
Cliente: /127.0.0.1
```

TERMINAL 2 · CLIENTE

```
$ java Cliente
> ola mundo
Servidor: OLA MUNDO
> SAIR
Servidor: TCHAU
```

1

Compilar os dois arquivos

2

Iniciar o servidor (ele espera)

3

Iniciar o cliente e digitar mensagens

Demonstração ao vivo e observação da rede

Roteiro

- 1 Iniciar o servidor e verificar a porta em LISTEN
- 2 Iniciar o cliente e ver a conexão ESTABLISHED
- 3 Enviar mensagens e observar as respostas
- 4 Encerrar com SAIR
- 5 Iniciar o cliente sem servidor: ConnectException
- 6 Trocar o cliente por nc ou telnet: é só texto

OBSERVANDO A REDE

```
# porta em LISTEN / ESTABLISHED
netstat -an | findstr 5050    (Windows)
ss -tan | grep 5050          (Linux)
lsof -i :5050                 (macOS)
```

```
# cliente alternativo, texto puro
nc localhost 5050
```

```
# Wireshark, interface loopback
tcp.port == 5050
```

Conexão com as aulas 1 e 2: observe o handshake TCP.

Limitações da POC



Um cliente por vez

accept() é chamado uma única vez; um segundo cliente fica esperando.



Sem tratamento de erros

Porta ocupada, servidor fora do ar ou queda de conexão encerram o programa com uma exceção.



Protocolo mínimo

Sem comandos, cabeçalhos ou versões. Qualquer texto é aceito.



Sem segurança

Texto puro na rede, sem autenticação nem criptografia.

Cada limitação é um ponto de partida para a aula 4.

Resumo da aula



Cliente-servidor: processos com papéis distintos, quem pede e quem atende.



Socket: extremidade de comunicação, identificada por IP e porta.



TCP e UDP: conexão confiável ou datagramas leves.



Em Java: ServerSocket, Socket e streams de entrada e saída.



Protocolo de aplicação: as regras que dão sentido aos bytes.

PARA LEVAR

**Socket =
IP + porta.**

**O resto é
protocolo de
aplicação.**

Próximos passos

AULA 4 · EVOLUINDO A POC

- Vários clientes ao mesmo tempo (threads)
- Protocolo com comandos e respostas de erro
- Tratamento de exceções
- Interface gráfica com JavaFX

ATIVIDADE SUGERIDA

- Executar a POC em duas máquinas da mesma rede
- Acrescentar o comando HORA: o servidor devolve a hora atual
- Capturar o tráfego no Wireshark e localizar o handshake TCP